Awesome! Let's build the "brain" of your logistics system.

For a 6th-semester project, your database needs to be relational (SQL is highly recommended here, like MySQL or PostgreSQL). You need to show how data connects through **Primary Keys (PK)** and **Foreign Keys (FK)**.

Here are the 4 core tables you need to make the Website, Desktop App, and Mobile App work together seamlessly.

1. The Users Table (One table to rule them all)

Instead of making separate tables for customers, drivers, and admins, use one table with a role column. This makes your login API much easier to write.

Column Name	Data Type	Description
user_id (PK)	INT	Unique ID for every user.
full_name	VARCHAR	Name of the person.
email	VARCHAR	For login.
password_hash	VARCHAR	Encrypted password (never store plain text!).
phone_number	VARCHAR	Crucial for drivers to call customers.
role	ENUM	'Admin', 'Driver', 'Customer', 'Warehouse'

2. The Hubs Table (The Warehouses)

Parcels move from one physical location to another. You need to know where a package is currently sitting.

Column Name	Data Type	Description
hub_id (PK)	INT	Unique ID for the warehouse.

hub_name	VARCHAR	e.g., "North City Hub", "Main Sorting Center".
city	VARCHAR	Location of the hub.

3. The Parcels Table (The Core Object)

This is the most important table. Every time a customer books a shipment on the **Website**, a row is created here.

Column Name	Data Type	Description
tracking_number (PK)	VARCHAR	e.g., "PK-10582" (This is what the Barcode prints!).
sender_id (FK)	INT	Links to Users table (the customer).
receiver_name	VARCHAR	Who is getting the box.
delivery_address	TEXT	Where the driver needs to go.
current_status	ENUM	'Pending', 'In_Transit', 'Out_for_Delivery', 'Delivered'
current_hub_id (FK)	INT	Links to Hubs. Where is the box right now?
driver_id (FK)	INT	Links to Users. Which driver has it in their van?

4. The Tracking_History Table (The Visual Timeline)

This is what makes the system look professional. Every time a barcode is scanned by the **Desktop App** or marked delivered by the **Mobile App**, you insert a new row here. The Website pulls from this table to show the timeline to the customer.

Column Name	Data Type	Description
history_id (PK)	INT	Unique ID for the event.
tracking_number (FK)	VARCHAR	Links to Parcels. Which box was scanned?
status_message	VARCHAR	e.g., "Arrived at Main Sorting Center" or "Dispatched".
updated_by (FK)	INT	Links to Users. Which worker/driver scanned it?
timestamp	DATETIME	The exact time it was scanned.

How the Apps Interact with this Database:

- **The Desktop App (Warehouse):** An admin types in a tracking_number (or uses a barcode scanner to type it instantly). The app sends an API request to update the current_status in the Parcels table and inserts a new row into Tracking_History.
- **The Mobile App (Driver):** The driver logs in. The app queries the Parcels table: *"Select all parcels where driver_id = X and status = 'Out_for_Delivery'"*. When they drop off the box, they click a button that updates the status to 'Delivered'.
- **The Website (Customer):** The customer types their tracking_number into a search bar. The website queries the Tracking_History table and displays the timeline of their package beautifully on the screen.

Would you like me to suggest the best tech stack (languages and frameworks) for building the shared API backend that will connect all three of these platforms?